جامعة الأزهر - غزة
Al Azhar University - Gaza
FACULTY OF ENGINEERING AND INFORMATION TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

FINAL REPORT [FYP I]

FURSA PLATFORM

A Digital Employment Platform for Gaza

STUDENT NAME 20240001
SUPERVISED BY
Dr. Name F. Family Name

JANUARY 2026
SEMESTER I 2025/2026

FINAL YEAR PROJECT REPORT
IT 0000 (COURSE NUMBER)

Fursa Platform

A Digital Employment Platform for Gaza Strip

by

Student Name 20240001

Supervised by
Dr. Name F. Family Name

In partial fulfillment of the requirement for the
Bachelor of Computer Science

Department of Computer Science and Information Technology
Faculty of Engineering and Information Technology

January 2026
Semester I 2025/2026

CERTIFICATE OF ORIGINALITY

This is to certify that I am responsible for the work submitted in this project, that the original work is my own except as specified in the references and acknowledgements, and that the original work contained herein have not been taken or done by unspecified sources or persons.

Student Name 20240001

ACKNOWLEDGEMENT

Praise and thanks to Allah first and foremost, whose blessing enabled the completion of this project.

I would like to express my sincere gratitude to my supervisor for their continuous guidance, encouragement, and support throughout the development of this project. Their expertise and valuable feedback were instrumental in shaping this work.

Special thanks go to my family for their patience and moral support, and to all colleagues who contributed ideas and feedback during the development process.

This project is dedicated to the people of Gaza, with hope that technology can contribute to economic empowerment and opportunity even in the most challenging circumstances.

ABSTRACT

The Fursa Platform is a full-stack web application designed to address the lack of an organized digital employment marketplace in the Gaza Strip. The platform aims to replace the chaotic and unstructured use of social media groups for job postings by providing a centralized, professional, and secure environment that connects job seekers with employers.

The system supports three user roles: Job Seekers, Employers, and Administrators. Job seekers can browse available opportunities, filter by specialization or work type (online/field), upload their CVs, and apply directly through the platform. Employers can post job opportunities with full descriptions and requirements, then review and respond to applications. Administrators oversee all posted opportunities through an approval workflow to ensure content quality and prevent spam.

The platform is built entirely in TypeScript using React with Vite for the frontend, Node.js with Express 5 for the backend API, and PostgreSQL as the relational database managed via Drizzle ORM. Key features include Clerk-based authentication with role-based access control, CV file upload via cloud object storage using signed URLs, and TanStack Query for efficient client-side data fetching and caching. The system is designed with Gaza's connectivity challenges in mind, prioritizing mobile-first design, RTL Arabic language support, and performance optimization for low-bandwidth environments.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE
1	INTRODUCTION	1
1.1	Preamble	1
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	Project Scope	3
1.4.1	Area of Specification	3
1.4.2	Targeted Users	3
1.4.3	Specific Platform	4
1.5	Project Stages	4
1.6	Expected Output	5
2	SYSTEM ANALYSIS	6
2.1	System Requirements	6
2.2	Functional Requirements	6
2.3	Non-Functional Requirements	8
2.4	User Types and Permissions	9
3	SYSTEM DESIGN	11
3.1	System Architecture	11
3.2	Technology Stack	12
3.3	Database Design	13
3.4	API Design	16
3.5	Security Design	20
4	PROJECT IMPLEMENTATION PLAN	22
4.1	Development Phases	22
4.2	Project Timeline	24
	REFERENCES	26
	APPENDICES	27

LIST OF TABLES

TABLE NO.	TITLE	PAGE
1.1	Project General Information	1
2.1	Functional Requirements — Job Seeker	7
2.2	Functional Requirements — Employer	7
2.3	Functional Requirements — Administrator	8
2.4	Non-Functional Requirements	9
2.5	User Roles and Permissions Matrix	10
3.1	Technology Stack Summary	12
3.2	User Table Fields	13
3.3	Job Table Fields	14
3.4	Application Table Fields	15
3.5	Auth API Endpoints	16
3.6	Jobs API Endpoints	17
3.7	Applications API Endpoints	18
3.8	Admin API Endpoints	19
3.9	Security Mechanisms	21
4.1	Development Phases and Tasks	22

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
3.1	System Architecture Diagram	11
3.2	Database Entity-Relationship Diagram	13
3.3	User Authentication Flow	20
3.4	Job Posting and Approval Workflow	21
4.1	Project Gantt Chart	25

CHAPTER 1
INTRODUCTION

1.1 Preamble

The job market in the Gaza Strip suffers from a significant lack of organized digital infrastructure. Job seekers and employers currently rely heavily on informal channels such as WhatsApp groups and Facebook communities, which are unstructured, difficult to search, and result in missed opportunities due to information overload and disorganization.

The Fursa Platform ("Fursa" means "Opportunity" in Arabic) is proposed as a solution to this problem. It is a full-stack web application designed specifically for the Gaza context, providing a structured, professional environment where employers can post legitimate job opportunities and job seekers can discover, filter, and apply for them in an organized manner.

Project Name	Fursa Platform — Digital Employment Platform for Gaza
Project Type	Full-Stack Web Application
Primary Language	TypeScript (Frontend + Backend)
Target Area	Gaza Strip, Palestine
Project Status	Planning and Analysis Phase
Academic Year	Semester I — 2025/2026

Table 1.1: Project General Information

1.2 Problem Statement

The absence of a dedicated digital employment platform in Gaza forces both employers and job seekers to use social media groups as their primary means of communication. This approach has several significant drawbacks:

- Job postings are mixed with unrelated content, making them hard to find.
- There is no standard format for job postings, leading to incomplete information.
- Applicants cannot formally apply; they must contact employers individually.
- Employers receive applications through private messages with no tracking system.
- There is no quality control mechanism to verify the authenticity of postings.
- Freelance and remote opportunities are not distinguished from field-based work.

1.3 Project Objectives

The primary objectives of the Fursa Platform are:

1. To develop a centralized web platform that aggregates all job opportunities in one organized location.
2. To provide structured job posting forms ensuring complete and standardized information.
3. To enable direct CV upload and application submission through the platform.
4. To implement a filtering and search system by specialization, work type, and keywords.

5. To build role-specific dashboards for job seekers, employers, and administrators.
6. To implement an admin approval workflow to ensure content quality and prevent spam.
7. To provide real-time notifications for key events such as applications and approvals.
8. To design the platform with consideration for Gaza's connectivity challenges.

1.4 Project Scope

1.4.1 Area of Specification

The Fursa Platform focuses on the labor market in the Gaza Strip, covering both online (remote) work opportunities and field-based (on-site) employment. The platform targets informal and semi-formal job markets, including freelance projects, part-time work, full-time employment, and short-term tasks. It does not cover government employment or formal corporate HR systems.

1.4.2 Targeted Users

The platform targets three distinct user groups:

- Job Seekers: Individuals looking for employment, freelance projects, or part-time work opportunities within or related to Gaza.
- Employers: Individuals, small businesses, NGOs, or companies seeking to hire talent or post work opportunities.
- Administrators: Platform moderators responsible for content quality, user management, and system oversight.

1.4.3 Specific Platform

The Fursa Platform is a web-based application accessible via standard desktop and mobile web browsers. It does not require installation. The system is built as a Single Page Application (SPA) using React with Vite, ensuring fast navigation and smooth user experience. The backend is a RESTful API built with Node.js and Express 5, with data stored in PostgreSQL via Drizzle ORM. Authentication is fully managed by Clerk. The platform supports Arabic (RTL) as the primary language with English support.

1.5 Project Stages

The project is divided into the following development stages:

1. Requirements Analysis: Gathering and documenting all system requirements and user needs.
2. System Design: Database schema design, API design, UI/UX wireframing, and architecture planning.
3. Backend Development: Building the REST API, authentication system, and database layer.
4. Frontend Development: Building the React interface with Clerk auth, role-based dashboards, and user interactions.
5. Integration and Testing: Connecting frontend and backend, unit testing, and system testing.
6. Deployment: Deploying the application to cloud hosting platforms.

1.6 Expected Output

The expected output of this project is a fully functional web application — the Fursa Platform — that is accessible via any modern web browser. The system will include a public-facing job listings page, user registration and

authentication, role-specific dashboards, a job posting and application workflow, an admin moderation panel, a real-time notification system, and CV upload and management functionality.

CHAPTER 2
SYSTEM ANALYSIS

2.1 System Requirements

System requirements for the Fursa Platform are divided into functional requirements — which describe what the system must do — and non-functional requirements — which describe how the system should perform. Requirements were gathered through analysis of existing informal job-sharing practices in Gaza and comparison with similar employment platforms.

2.2 Functional Requirements

2.2.1 Job Seeker Requirements

FR-JS-01	The system shall allow job seekers to register and sign in via Clerk (email/password or OAuth). Profile details (phone, location, bio) are completed during onboarding.
FR-JS-02	The system shall allow job seekers to upload a CV in PDF, DOC, or DOCX format (max 5 MB).
FR-JS-03	The system shall display all approved job opportunities with title, type, category, and posting date.
FR-JS-04	The system shall allow filtering by work type (online/field/hybrid) and specialization category.
FR-JS-05	The system shall allow keyword search across job title and description.
FR-JS-06	The system shall allow job seekers to apply to a job with an optional cover letter.
FR-JS-07	The system shall allow job seekers to save jobs to a personal bookmarks list.
FR-JS-08	The system shall provide a dashboard showing application history and their current status.
FR-JS-09	The system shall send a notification when an application is accepted or rejected.

Table 2.1: Functional Requirements — Job Seeker

2.2.2 Employer Requirements

FR-EM-01	The system shall allow employers to register via Clerk and select the Employer role during onboarding.
FR-EM-02	The system shall allow employers to post a new job with title, description, requirements, type, category, contact info, and deadline.
FR-EM-03	The system shall set new job postings to "Pending" status awaiting admin approval.
FR-EM-04	The system shall allow employers to edit a pending job posting before approval.
FR-EM-05	The system shall display all applications received for each posted job.
FR-EM-06	The system shall allow employers to download or view the CV of each applicant.

FR-EM-07	The system shall allow employers to accept or reject individual applications.
FR-EM-08	The system shall allow employers to close a job opportunity at any time.
FR-EM-09	The system shall notify the employer when a new application is received.

Table 2.2: Functional Requirements — Employer

2.2.3 Administrator Requirements

FR-AD-01	The system shall provide an admin dashboard accessible only to administrator accounts.
FR-AD-02	The system shall display all pending job postings for admin review.
FR-AD-03	The system shall allow the admin to approve a pending job, making it visible to all users.
FR-AD-04	The system shall allow the admin to reject a pending job with a mandatory reason.
FR-AD-05	The system shall notify the employer of approval or rejection with the reason included.
FR-AD-06	The system shall allow the admin to view all registered users.
FR-AD-07	The system shall allow the admin to deactivate or reactivate any user account.
FR-AD-08	The system shall display platform statistics: total users, jobs, applications, approval rate.

Table 2.3: Functional Requirements — Administrator

2.3 Non-Functional Requirements

ID	Requirement	Priority
NFR-01	The system shall load the job listings page within 3 seconds on a 3G connection.	High
NFR-02	The system shall support a minimum of 500 concurrent users without performance degradation.	Medium
NFR-03	User authentication and credential management shall be fully delegated to Clerk, ensuring industry-standard security practices including encrypted password storage and secure session handling.	High
NFR-04	The system shall use Clerk session tokens for all authenticated API requests; session lifetime is managed by Clerk.	High
NFR-05	The platform shall be fully responsive and usable on screens from 320px width.	High
NFR-06	The Arabic RTL layout shall render correctly on all major browsers.	High
NFR-07	File uploads shall be restricted to PDF/DOC/DOCX with a maximum size of 5 MB.	High
NFR-08	API endpoints shall implement rate limiting to prevent brute-force attacks.	Medium

NFR-09	The system shall achieve 99% uptime on the hosting platform.	Medium
NFR-10	All user inputs shall be validated on both client and server side.	High

Table 2.4: Non-Functional Requirements

2.4 User Types and Permissions

The system implements a role-based access control (RBAC) model with three distinct roles. Each role has clearly defined permissions that are enforced at the API level through middleware.

Feature / Action	Job Seeker	Employer
Register / Login	Yes	Yes
Browse Approved Jobs	Yes	Yes
Search and Filter Jobs	Yes	Yes
View Job Details	Yes	Yes
Upload CV	Yes	No
Apply to Jobs	Yes	No
Save / Bookmark Jobs	Yes	No
View My Applications	Yes	No
Post New Job	No	Yes
Manage My Job Posts	No	Yes
View Job Applicants	No	Yes
Accept / Reject Applications	No	Yes
Close Job Posting	No	Yes
Admin Dashboard	No	No
Approve / Reject Jobs	No	No
Manage All Users	No	No

Table 2.5: User Roles and Permissions Matrix (Admin has full access to all)

CHAPTER 3
SYSTEM DESIGN

3.1 System Architecture

The Fursa Platform follows a client-server architecture with a clear separation between the frontend, backend, and data layers. The frontend is a React SPA that communicates with the backend exclusively through a RESTful JSON API generated from an OpenAPI specification. Authentication is handled entirely by Clerk on both client and server sides.

Figure 3.1 below illustrates the overall system architecture. The client (browser) sends HTTP requests to the Express 5 API server. The server validates Clerk session tokens via middleware, resolves the user role, queries PostgreSQL through Drizzle ORM, and issues signed URLs for secure file operations against cloud object storage. Real-time notifications are deferred to a future phase.

Client Layer	React + Vite (SPA, TypeScript) — Runs in the user's browser. Communicates via HTTP/HTTPS and WebSocket.
API Layer	Node.js + Express.js — RESTful API server. Handles authentication, business logic, and data operations.
Real-time Layer (Deferred)	Not implemented in current scope — planned for a future release.
Database Layer	PostgreSQL with Drizzle ORM — Stores users, jobs, and applications in a relational schema.
File Storage	Cloud Object Storage — CV uploads use a signed URL flow; the client uploads directly to object storage, and the server stores the resulting URL reference.
Email Service	Deferred — Both real-time notifications and email are out of scope for the current phase.
Hosting	Deployed as a unified monorepo on Replit (pnpm workspace); frontend at /, API at /api/*.

Figure 3.1: System Architecture Layers

3.2 Technology Stack

The entire application is built using TypeScript on both frontend and backend, enabling type safety and code sharing across the monorepopoliftying the development workflow. The following table summarizes all technologies used:

Layer	Technology	Purpose
Frontend	React + Vite (TypeScript)	UI framework and build tool
Styling	Tailwind CSS v4	Utility-first responsive design (v4 engine)
State Mgmt.	Zustand	Global state management
HTTP Client	TanStack Query	Server state management, caching, and API data

		fetching
Backend	Node.js + Express 5	REST API server
Database	PostgreSQL + Drizzle ORM	Relational database and query builder / ORM
Identity Management	Clerk (@clerk/express, @clerk/react)	Managed auth: sign-up, sign-in, session tokens, OAuth
File Upload	Object Storage SDK + Signed URLs	Direct-to-cloud CV upload via pre-signed URLs
Real-time	— (Deferred)	Planned for future phase
Email	— (Future Phase)	Email notifications deferred to a future development phase
Validation	Zod (auto-generated from OpenAPI spec)	Type-safe request/response validation
Security	Helmet.js	Secure HTTP headers (passwords managed by Clerk)
Rate Limiting	express-rate-limit	Brute-force protection
Routing	Wouter	Lightweight client-side routing
UI Components	shadcn/ui	Accessible, composable React component library
API Contract	OpenAPI + Zod	Spec-first API; auto-generates Zod schemas and React Query hooks

Table 3.1: Technology Stack Summary

3.3 Database Design

The database is implemented using PostgreSQL, a robust relational database management system. Three primary tables are used: users, jobs, and applications. The schema design follows Drizzle ORM conventions with typed column definitions, foreign key constraints, and explicit relations between tables.

3.3.1 User Schema

Field	Type / Constraints	Description
id	Serial / Integer (auto-increment)	Unique identifier
name	String, required	Full name of the user
email	String, required, unique	Email address — synced from Clerk identity on first login
clerkId	String, required, unique	Clerk identity provider ID — links platform user to Clerk

		account
role	Enum: seeker / employer / admin	User type, default: seeker
phone	String, optional	Contact phone number
location	String, optional	City or area within Gaza
bio	String, optional	Short personal description
cvUrl	String, optional	Server storage URL of uploaded CV
savedJobs	Managed via separate savedJobs table (FK → jobs.id)	Bookmarked jobs — stored in dedicated savedJobs join table
isActive	Boolean, default: true	Account active status
createdAt	Date (auto)	Account creation timestamp
updatedAt	Date (auto)	Last update timestamp

Table 3.2: User Table Fields

3.3.2 Job Schema

Field	Type / Constraints	Description
id	Serial / Integer (auto-increment)	Unique identifier
title	String, required	Job title
description	String, required	Full job description
requirements	String, optional	Skills and experience required
type	Enum: online/field/hybrid	Work location type
category	String, required	Specialization category
contactInfo	String, required	How to contact the employer
employer	Integer (FK → users.id), required	Reference to posting employer
status	Enum: pending/approved/rejected	Admin approval status, default: pending
rejectionReason	String, optional	Populated when status=rejected
isOpen	Boolean, default: true	Whether job is still accepting applications
deadline	Date, optional	Application deadline
createdAt	Date (auto)	Posting timestamp

Table 3.3: Job Table Fields

3.3.3 Application Schema

Field	Type / Constraints	Description
id	Serial / Integer (auto-increment)	Unique identifier

job	Integer (FK → jobs.id), required	Reference to the applied job
applicant	Integer (FK → users.id), required	Reference to the job seeker
coverLetter	String, optional	Optional cover letter text
cvUrl	String, optional	CV URL for this specific application
status	Enum: pending/accepted/rejected	Application status, default: pending
seenByEmployer	Boolean, default: false	Whether employer viewed the application
createdAt	Date (auto)	Application submission timestamp

Table 3.4: Application Table Fields

3.4 API Design

The Fursa Platform exposes a RESTful API following standard HTTP conventions. All endpoints return JSON responses with a consistent structure containing "success" (boolean), "message" (string), and "data" (object or array) fields. Authentication is handled via Clerk session tokens; the server middleware validates the token and resolves the current user and role before any route handler executes.

3.4.1 Authentication Endpoints

Method + Endpoint	Description	Auth Required
POST /api/auth/onboarding	Bootstrap user row from Clerk identity on first login; assign role and profile	None
GET /api/me	Get current authenticated user profile and role	None
PUT /api/me/profile	Update profile information (phone, location, bio)	Any user
POST /api/storage/upload-url	Request a signed URL for direct CV upload to object storage	Any user
PUT /api/me/cv	Save CV object storage URL to user profile after upload completes	Seeker

Table 3.5: Auth API Endpoints

3.4.2 Job Endpoints

Method + Endpoint	Description	Auth Required
GET /api/jobs	Get all approved jobs with filters and pagination	None
GET /api/jobs/:id	Get full details of a specific job	None
POST /api/jobs	Create a new job posting (status: pending)	Employer
PUT /api/jobs/:id	Edit a job (only before admin approval)	Employer (owner)
DELETE /api/jobs/:id	Delete a job posting	Employer or Admin

GET /api/jobs/my-jobs	Get all jobs posted by current employer	Employer
POST /api/jobs/:id/save	Toggle save/unsave a job	Seeker
GET /api/jobs/saved	Get list of saved jobs	Seeker

Table 3.6: Jobs API Endpoints

3.4.3 Application Endpoints

Method + Endpoint	Description	Auth Required
POST /api/applications/:jobId	Submit an application to a job	Seeker
GET /api/applications/my	Get all my applications as a seeker	Seeker
GET /api/applications/job/:jobId	Get all applicants for a specific job	Employer (owner)
PUT /api/applications/:id/status	Accept or reject an application	Employer (owner)

Table 3.7: Applications API Endpoints

3.4.4 Admin Endpoints

Method + Endpoint	Description	Auth Required
GET /api/admin/jobs/pending	List all pending jobs for review	Admin
PUT /api/admin/jobs/:id/approve	Approve a pending job posting	Admin
PUT /api/admin/jobs/:id/reject	Reject with mandatory reason	Admin
GET /api/admin/users	Get all registered users	Admin
PUT /api/admin/users/:id/toggle	Activate or deactivate a user	Admin
GET /api/admin/stats	Get platform statistics dashboard data	Admin

Table 3.8: Admin API Endpoints

3.5 Security Design

The Fursa Platform implements a defense-in-depth security approach with multiple independent layers of protection. No single security failure should be sufficient to compromise the system:

Security Layer	Mechanism	Protection Target
Identity Management	Clerk (OAuth, email/password, session tokens)	Secure sign-in, session management, credential storage
Authentication Middleware	Clerk session token validation on every API request	Unauthorized API access

Authorization	Role middleware on all routes	Privilege escalation
File Upload	Type + size validation; signed URLs expire after 15 minutes	Malicious file uploads
Input Validation	Joi schemas on all endpoints	Injection attacks
CORS Policy	Whitelist of allowed origins	Cross-origin attacks
Rate Limiting	express-rate-limit on login	Brute-force attacks
HTTP Headers	Helmet.js security headers	XSS and clickjacking
Content Approval	Admin review workflow	Spam and abuse

Table 3.9: Security Mechanisms

CHAPTER 4
PROJECT IMPLEMENTATION PLAN

4.1 Development Phases

The development is organized into five sequential phases spanning approximately nine weeks. Each phase has clear deliverables and success criteria before proceeding to the next phase.

Phase	Key Tasks	Duration
Phase 1: Infrastructure Setup	pnpm monorepo init, React/Vite + Express 5 setup, PostgreSQL + Drizzle ORM, Clerk integration (frontend + backend), Object Storage config, CORS and Helmet setup.	Weeks 1-2
Phase 2: Core Backend	Clerk auth middleware + user auto-bootstrapping on first login, Job CRUD APIs with pending workflow, Application APIs, Signed URL file upload flow.	Weeks 3-4
Phase 3: Core Frontend	Job listing page with filters, Job detail page, Application form with CV upload, Registration and login pages.	Weeks 4-5
Phase 4: Dashboards	Seeker dashboard (applications + saved jobs), Employer dashboard (my jobs + applicants), Admin dashboard (review queue + stats).	Weeks 6-7
Phase 5: Notifications and Deployment	Final UI polish, accessibility and RTL testing, bug fixes, Deployment to Replit.	Weeks 8-9

Table 4.1: Development Phases and Tasks

4.2 Project Timeline

The following table summarizes the project timeline across all phases and weeks:

Week	Activities	Deliverable
Week 1	Environment setup, project structure, Git repository	Monorepo scaffold with DB + Clerk + Object Storage connected
Week 2	User bootstrapping from Clerk identity, auth middleware, role system	Working Clerk-authenticated API with role enforcement
Week 3	Job model, CRUD API, CV upload via signed URL flow	Full job management backend
Week 4	Application API + React project + routing setup	Application backend + React skeleton
Week 5	Job listing page, filtering, search, job detail page	Public-facing frontend complete
Week 6	Seeker and Employer dashboards	User dashboards functional
Week 7	Admin dashboard: approval queue, user management, stats	Admin panel complete

Week 8	Final UI polish, RTL/accessibility testing, deployment	Real-time features integrated
Week 9	Final testing, bug fixes, deployment, documentation	Live deployed application

Figure 3.1 (Appendix A): Full Gantt Chart available in Appendices

REFERENCES

[1] Codevolution. (2024). Node.js, Express, PostgreSQL & React — Full Stack Development. YouTube Series. https://www.youtube.com/@Codevolution

[2] Mozilla Developer Network. (2024). Express.js Documentation. https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs

[3] The PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. https://www.postgresql.org/docs/

[4] Meta Platforms, Inc. (2024). React Documentation. https://react.dev/

[5] Tailwind Labs. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs/

[6] TanStack. (2024). TanStack Query Documentation. https://tanstack.com/query/latest/docs/framework/react/overview

[7] Drizzle Team. (2024). Drizzle ORM Documentation. https://orm.drizzle.team/docs/overview

[8] Clerk. (2024). Clerk Documentation — Authentication and User Management. https://clerk.com/docs

[9] OWASP. (2024). OWASP API Security Top 10. https://owasp.org/www-project-api-security/

[10] Palestinian Central Bureau of Statistics. (2023). Labor Force Survey Annual Report. PCBS, Ramallah.

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
A	Project Gantt Chart	27
B	Database ER Diagram	28
C	API Request/Response Examples	29
D	Environment Variables Reference	31
E	Project Folder Structure	32

APPENDIX A: Project Gantt Chart

Task / Week	Weeks 1-4	Weeks 5-9
Infrastructure Setup	Weeks 1-2 ####	
User Auth Backend	Weeks 2-3 ####	
Job + App Backend	Weeks 3-4 ####	
Frontend Core	Week 4 ##	Week 5 ###
Dashboards		Weeks 6-7 ####
Notifications		Week 8 ###
Testing + Deploy		Week 9 ###

APPENDIX E: Project Folder Structure

```
fursa/                              # pnpm monorepo root
├── artifacts/
│   ├── fursa/                      # React frontend (Vite + TypeScript)
│   │   └── src/
│   │       ├── App.tsx             # Clerk provider, router, role guards
│   │       ├── lib/i18n.tsx        # Arabic/English translations + RTL
│   │       ├── pages/              # Home, Jobs, JobDetail, Onboarding,
Dashboards
│   │       └── components/         # Shared UI components (shadcn/ui
based)
│   └── api-server/                 # Express 5 backend (TypeScript)
│       └── src/
│           ├── middlewares/        # auth.ts (requireAuth,
loadCurrentUser, requireRole)
│           └── routes/             # publicJobs, me, seeker, employer,
admin, storage
├── lib/
│   ├── db/src/schema/              # Drizzle tables: users, jobs,
applications, savedJobs
│   ├── api-spec/                   # OpenAPI specification
│   ├── api-zod/                    # Auto-generated Zod schemas from
OpenAPI
│   ├── api-client-react/           # Auto-generated TanStack Query hooks
│   └── object-storage-web/         # Signed URL upload client
├── scripts/src/seedFursa.ts        # Seeds demo users, jobs, applications
├── pnpm-workspace.yaml
└── .env                            # CLERK_*, DATABASE_URL,
OBJECT_STORAGE_*
```